高频交易快照的量化因子 MapReduce 计算：设计、实现与优化

关新灏 12311027　孙祺 12311019

2025 年 12 月 22 日

目录

1 问题描述与任务理解 2
1.1 任务目标 2
1.2 因子计算口径与数值稳定 2
2 数据与输出格式 2
2.1 输入数据形态 2
2.2 输出组织 2
3 难点分析 2
3.1 难点 1：小文件数量多导致 Mapper 过多 2
3.2 难点 2：Shuffle/Sort 触发的 spill/merge 引发本地 IO 放大 2
3.3 难点 3：写表头与输出文件组织和 reducer 并行度冲突 3
3.4 难点 4：字符串解析与对象创建带来的 GC/CPU 压力 3
4 整体技术方案（含图示） 3
4.1 方案概述 3
4.2 数据流架构图 4
4.3 关键热点与优化点示意 4
5 代码模块化设计思路 4
5.1 模块划分总览 4
5.2 Snapshot：避免 split 的解析设计 4
5.3 FactorCalculator：纯函数化计算 5
5.4 FactorAggWritable + Combiner：减少 Reduce 端解析与 merge 5
5.5 FactorMapper：过滤与 key 设计 5
5.6 TradingDayPartitioner：按交易日分区 6
5.7 FactorReducer：写表头 + 输出文件组织 6
5.8 CalcDriver：参数配置与 autoTune 6
6 课程感想与大模型使用体会 6

1 问题描述与任务理解

1.1 任务目标

输入为多个交易日的股票快照数据（CSV），每行对应一个时刻的订单簿快照，包含交易日、时间戳、全市场累计量/加权价格以及前 5 档买卖盘价格与数量等字段。任务要求：

- 对每一条快照（时刻 t）计算 20 个 LOB 因子；
- 对同一交易日同一时刻（例如 `20240102,093000`）来自不同股票的因子向量做横截面平均；
- 输出为 CSV：首行表头 `tradeTime, alpha_1, ..., alpha_20`，后续每行以 `tradeTime` 开头，跟随 20 个均值因子（保留 6 位小数）。

1.2 因子计算口径与数值稳定

因子使用订单簿前 $N = 5$ 档数据，时间间隔参数 $\Delta t = 1$（即与上一条快照对比得到增量类特征）。实际计算中可能出现分母为 0 或极小值，采用

$$\frac{a}{b} \to \frac{a}{b+\varepsilon}, \quad \varepsilon = 10^{-7}$$

避免产生 NaN/Inf，保证输出稳定。

2 数据与输出格式

2.1 输入数据形态

本项目输入规模：

- 共 5 个交易日；
- 每天 60+ 只股票；
- 每只股票一个 CSV 文件，约 4900 行 × 30 列左右；
- 运行在 Docker 内的伪分布式（CPU 核数常见为 4 核，内存约 8–12GB+）。

2.2 输出组织

输出目标为按交易日/文件组织的 CSV。本文实现采用 `MultipleOutputs` 按交易日生成独立文件（如 `0102.csv`），并保证每个文件首行写表头，后续为各时刻均值行。

3 难点分析

3.1 难点 1：小文件数量多导致 Mapper 过多

当输入目录包含大量小文件时，默认 `TextInputFormat` 会为每个文件产生一个或多个 split，导致 Mapper 数量偏大，任务启动/调度开销上升。在 Docker/LocalJobRunner 下尤其明显：并行度受限但任务数激增，会出现 CPU 并未充分利用但总耗时增加的情况。

3.2 难点 2：Shuffle/Sort 触发的 spill/merge 引发本地 IO 放大

从优化前的 counters 可以较明确地定位瓶颈：在 shuffle/sort 阶段出现了大量 spill/merge，导致本地文件系统（FILE）读写量远超真正进入 reducer 的 shuffle 数据量，出现明显的 IO 放大效应。

- 优化前：本地 IO 显著高于 shuffle 规模优化前 reducer 的 shuffle 数据量为 `Reduce shuffle`

`bytes = 453,364,989`（约 453MB），但本地文件系统统计显示 `FILE: Number of bytes read = 4,524,854,003`(约 4.52GB),`FILE: Number of bytes written = 4,525,248,536`（约 4.53GB）。这意味着在 shuffle/sort 过程中，为完成排序归并产生了约 $4.52/0.453 \approx 10$ 倍的额外本地磁盘读写，典型表现为多轮段文件（segment）合并与落盘（spill）。

- 优化后：Combiner 显著减少进入 shuffle 的物化数据，但仍存在 spill/merge 开销在引入可聚合的 `Writable` 中间结构并启用 Combiner 之后，`Combine input records = 7,203,000` 被压缩为 `Combine output records = 225,694`,对应 `Reduce shuffle bytes = 32,864,287`（约 32.9MB），显著降低了需要跨阶段传输与排序的数据规模。同时，优化后 counters 仍显示 `Spilled Records = 451,388`，以及本地文件系统 `FILE: Number of bytes written = 1,008,399,839`（约 1.01GB），说明在本地模式/有限 sort buffer 下，Map/Reduce 仍可能触发若干轮 spill/merge；但相较优化前，进入 reducer 的 shuffle 数据规模已明显下降，瓶颈由“数据量过大”转向“局部 spill/merge 的实现与参数约束”。

3.3 难点 3：写表头与输出文件组织和 reducer 并行度冲突

若要在每个交易日文件首行写表头，最直观做法是 `setNumReduceTasks(1)` 保证顺序，但这样会放弃 reducer 并行度。本文方案选择：

- 通过 `Partitioner` 将同一天路由到同一个 reducer；
- reducer 内按日期切换时写表头；
- 输出按交易日拆分文件，兼顾输出格式与并行度。

3.4 难点 4：字符串解析与对象创建带来的 GC/CPU 压力

快照行字段较多，若每行使用 `String.split(",")` 会产生大量临时对象（数组 + 子串 + 装箱等）。在高频快照数据下这会显著增加 GC，导致 CPU 时间浪费。本文在 `Snapshot` 中采用手写扫描逗号与手写 parseLong 降低分配与解析开销。在进一步优化中，我们也将 Mapper 输出从字符串改为 `double[20]` 的数值缓冲写入，避免“Map 端拼接 CSV / 下游再 parse”的重复对象创建，从而进一步降低 GC 与 CPU 解析开销。

4 整体技术方案（含图示）

4.1 方案概述

整体采用 MapReduce 一阶段聚合（One-stage Aggregation）：

- Mapper：对每条快照构造 `Snapshot`，计算 20 因子；输出 key=`tradingDay,HHMMSS`；
- Value：Map 端直接产生 `double[20]` 因子向量,并写入自定义可聚合结构 `FactorAggWritable(sum[20], count)`（避免先拼接 CSV、再 parse 回 double 的往返开销）；
- Combiner：在 map 端按 key 预聚合（sum+count），显著减少 shuffle 数据与 reduce 端 value 条数；
- Reducer：按 key 汇总 sum 与 count，输出均值 CSV 行，并用 `MultipleOutputs` 按交易日写文件与表头。

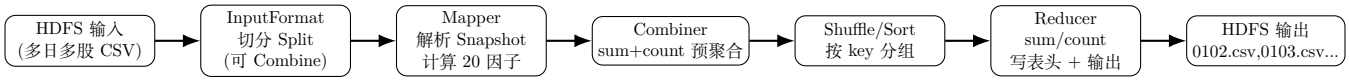

图 1. 总体数据流：Mapper 计算、Combiner 聚合、Reducer 输出按日文件

热点 A：字符串解析 `split(",")` 产生大量临时对象 → GC/CPU 压力大	优化：手写扫描 + parseLong
热点 B：Reducer 端解析与 merge/spill value 为 CSV 字符串 → 144 万次 parseDouble + merge 成本	优化：Writable + Combiner
热点 C：小文件过多 Mapper 数量爆炸 → 启动开销高、并行度受限	优化：CombineTextInputFormat
热点 D：Reducer 并行与输出样式冲突 Reducer 数量少, 并行度低 → Reduce 阶段缓慢	优化：解析日期作为 Reducer 映射依据

图 2. 性能瓶颈与对应优化点

4.2 数据流架构图

4.3 关键热点与优化点示意

5 代码模块化设计思路

5.1 模块划分总览

代码按“数据结构层—计算层—MapReduce 流水线层”组织：

- **数据结构层**：`Snapshot`（快照解析与字段存储）；
- **计算层**：`FactorCalculator`（纯数学计算 20 因子）；
- **流水线层**：
  - `FactorMapper`（过滤、构造快照、计算因子、输出聚合结构）；
  - `FactorAggWritable`（自定义中间结果：sum[20]+count）；
  - `FactorCombiner`（map 端预聚合，减少 shuffle）；
  - `TradingDayPartitioner`（按交易日分区）；
  - `FactorReducer`（汇总均值、写表头、MultipleOutputs 输出）；
  - `CalcDriver`（任务配置、autoTune、自适应参数与后处理重命名）。

5.2 Snapshot：避免 split 的解析设计

为降低 `split(",")` 带来的对象分配与 GC 压力，`Snapshot` 采用：

- 扫描逗号位置定位字段区间；
- 使用手写 `parseLongFast(String, start, end)` 将子串解析成 long；
- 仅提取因子计算所需字段:`tradingDay, tradeTime, tBidVol, tAskVol, wBidPrc, wAskPrc` 与前 5 档 `bp/bv/ap/av`。

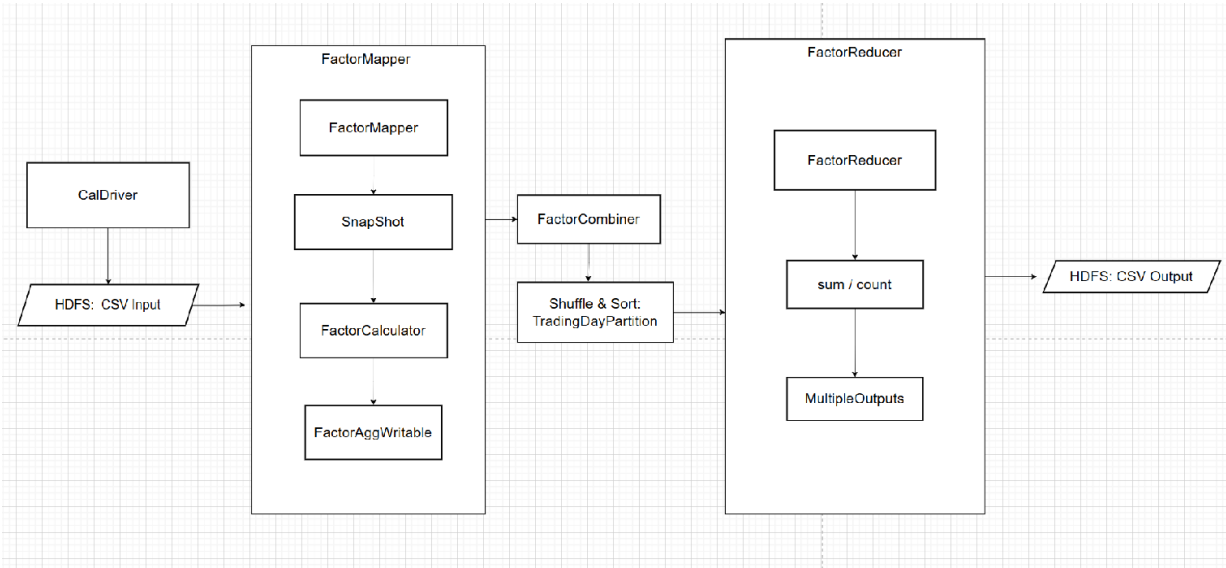

图 3. 模块流程图

5.3 FactorCalculator：纯函数化计算

`FactorCalculator.calculateAll(curr, prev, factorsBuf)` 仅依赖输入快照，将 20 个因子直接写入调用方传入的 `double[20]` 缓冲区，而不再返回字符串或重复分配新数组。其特点：

- 常量参数：$N = 5$，$\varepsilon = 10^{-7}$；
- 先将 long 转 double 参与计算，避免整数截断；
- 增量因子（如价量变化）依赖 `prev`，因此 Mapper 维护上一条快照用于 $\Delta t = 1$；
- 通过“写入 buffer”的方式减少对象创建与 GC 压力，使 Map 端计算更接近纯数值流水线。

5.4 FactorAggWritable + Combiner：减少 Reduce 端解析与 merge

中间结果为可聚合结构：

$$\texttt{FactorAggWritable} = \{\texttt{sum}[20],\ \texttt{count}\}$$

在当前实现中，Mapper 端将 `FactorCalculator` 计算得到的 `double[20]` 通过 `FactorAggWritable.setFromDou` 写入中间结果：其中 `sum[20]` 保存 20 个因子之和（初始即为本条记录的因子值），`count=1` 表示横截面计数。这样在 Combiner/Reducer 端只需做数组逐项累加与计数累加即可完成聚合。Combiner 与 Reducer 做相同的可交换/可结合运算：对同 key 的 `sum` 与 `count` 分别累加。这样可以：

- map 端先聚合多股票的同一时刻因子；
- 极大减少进入 reducer 的记录数；
- 降低 shuffle bytes、reduce merge/spill 的本地 IO 成本。

5.5 FactorMapper：过滤与 key 设计

Mapper 输出 key 采用 `tradingDay,HHMMSS`，理由：

- 能自然对应按日按时刻做横截面平均的聚合目标；
- reducer 可按 key 分组得到同一时刻所有股票的因子集合；

- 输出中仅需 `tradeTime`，因此 reducer 写出时可丢弃 `tradingDay`，用于文件名即可。

  同时 Mapper 保持交易时段过滤：

- `tradeTime < 092957`：丢弃或只用于缓存（视实现约束）；
- `tradeTime == 092957`：初始化 `prevSnapshot`，不输出；
- `tradeTime > 092957`：计算因子并输出。
  此外，Mapper 端实现中复用了两类缓冲对象以减少分配：一是 `double[20] factorsBuf` 作为因子计算输出缓冲区；二是复用单个 `FactorAggWritable outVal` 作为输出值对象（每条记录通过 `setFromDoubles` 覆盖写入）。这种写法配合 Writable 的序列化流程，可在保证语义正确的前提下减少 GC，提升吞吐。

5.6 TradingDayPartitioner：按交易日分区

分区器从 key 解析出 `YYYYMMDD` 的整数 day，取模映射到 reducer：

$$\texttt{partition} = day \bmod R$$

相比 `hashCode`，直接解析整数取模更加稳定、可预期，避免实现依赖和不必要的 hash 碰撞。

5.7 FactorReducer：写表头 + 输出文件组织

Reducer 内部逻辑：

- key 拆分得到 `day,time`；
- 当日期切换时写一次表头；
- 对 `FactorAggWritable` 的 sum/count 汇总求均值；
- 使用 `MultipleOutputs` 输出到 `MMDD.csv` 对应的文件流。

5.8 CalcDriver：参数配置与 autoTune

Driver 负责：

- 打开 `mapreduce.map.output.compress=true` 与 `SnappyCodec`（仅压缩字节，不影响数值精度）；
- 使用 `CombineTextInputFormat` 合并小文件 split（例如 64/32MB）；
- `autoTune(conf)`：根据 `availableProcessors()` 与 `Runtime.maxMemory()` 分档设置：
  - `mapreduce.local.map.tasks.maximum` / `mapreduce.local.reduce.tasks.maximum`(本地模式并发上限)；
  - `mapreduce.task.io.sort.mb`（sort buffer 大小，影响 spill 次数）；
  - `aqua.combine.max.mb/min.mb`（Combine split 的大小）；
  - 默认 reducer 数。

6 课程感想与大模型使用体会

在本次实验中，我们使用 AI 工具（如 ChatGPT/Codex）用于方案梳理、代码修改建议与排错思路的生成。总体而言，AI 在快速枚举可能的优化方向、解释 MapReduce 机制（Shuffle/Sort、Combiner、InputSplit 等）方面具有明显效率优势，能够缩短从问题到候选方案的探索时间。然

而，AI 给出的建议往往是通用模式，是否适用于当前实现仍需依赖实验数据验证；若缺少日志与指标支撑，容易出现修改幅度大但收益不明显，甚至引入新问题的情况。因此，本次实验也强化了我们对工程实践中的一些认识：先定位瓶颈，再针对性修改，最后用同一组可量化指标闭环验证。

尤其重要的是日志与 Counters 的作用。MapReduce 的性能瓶颈通常并不直观，仅从代码结构难以判断是计算受限、GC 受限还是 IO/Shuffle 受限。我们通过 Hadoop Job 的 counters（例如 `Reduce shuffle bytes`、`FILE bytes read/write`、`Spilled Records`、`Merged Map outputs`、`Combine input/output records` 等）对执行过程进行分析，能够将“慢”具体归因到某个阶段：例如 Reduce 端的归并排序与 spill 会造成明显的本地 IO 放大；Combiner 若未生效则会导致进入 Reduce 的记录数与网络/磁盘开销维持高位；小文件导致 Mapper 过多则会引发额外的启动与调度成本。